MSc in Data Science

Data Analytics & Algorithms

Continuous Assessment

FaceTrace: AI-Powered Reverse Face Search System

Module	Data Analytics & Algorithms
Submission Date	April 13, 2026
Group members	Anjaly , Surekha
Dataset	LFW (Labeled Faces in the Wild) — Top 50
Algorithm	Convolutional Neural Network (CNN)

1. Product Definition & Sustainability

1.1 Product Overview

FaceTrace is an AI-powered reverse face search web application. The system allows a user to upload any photograph containing a human face and searches a locally-indexed database of face images to find visually similar matches, ranked by percentage similarity. It replicates the core functionality of commercial tools such as PimEyes, but operates entirely on local infrastructure with no dependency on external cloud services.

The application is built using a Convolutional Neural Network (CNN) implemented from scratch using NumPy, combined with OpenCV for face detection and a Flask REST API backend. The frontend is a single-page HTML/CSS/JavaScript application served directly by the Flask server.

1.2 Sustainability & Resilience Objectives

The project addresses sustainability and resilience on multiple levels:

- Privacy-first design: All processing is performed locally on the user's machine. No images are uploaded to external servers, eliminating privacy risks associated with cloud-based facial recognition services.
- Accessibility: The system runs on low-cost consumer hardware (standard laptop/desktop CPU). No GPU is required, making it accessible to users and organisations in low-resource environments.
- Data sovereignty: Institutions such as hospitals, refugee agencies, or humanitarian organisations can use the system to verify identity without transmitting sensitive biometric data across the internet.
- Open source stack: The entire system uses open source libraries (Flask, OpenCV, NumPy, scikit-learn), ensuring no vendor lock-in and long-term maintainability.
- Resilience: The application functions fully offline with no internet dependency after initial setup, making it resilient to network outages.

1.3 Feasibility & Scope

The project scope was deliberately constrained to a locally-operated system to ensure feasibility within the academic timeframe. The LFW Top 50 dataset was selected as a manageable subset of the full Labeled Faces in the Wild dataset, providing sufficient data to demonstrate the core face search functionality. The primary technical challenge — building a CNN-based face embedding system from scratch — was successfully completed and deployed as a working web application.

2. Data Acquisition & Preparation

2.1 Dataset Selection & Sourcing

The Labeled Faces in the Wild (LFW) dataset was selected as the primary data source for this project. LFW is a publicly available benchmark dataset maintained by the University of Massachusetts Amherst, originally created for the study of unconstrained face recognition.

Property	Details
Full Dataset Size	13,233 images of 5,749 individuals
Subset Used	LFW Top 50 (top 50 most-photographed individuals)
Image Format	JPEG, 250x250 pixels
Source	University of Massachusetts — publicly available
License	Free for academic/non-commercial use
Ethics	All subjects are public figures; images sourced from news media

The Top 50 subset was chosen to balance dataset size with computational feasibility on consumer hardware. Each person in the subset has multiple photos, enabling meaningful testing of the similarity matching system.

2.2 Ethics & Compliance

The LFW dataset consists exclusively of images of public figures (politicians, athletes, celebrities) sourced from publicly available news photography. The use of this dataset for academic research is explicitly permitted under its terms of use. No private individuals are included. All processing is performed locally with no data transmitted externally. The system includes no capability for real-time surveillance or tracking.

2.3 Exploratory Data Analysis (EDA)

Prior to model development, an exploratory analysis of the dataset was conducted:

- Distribution of images per person: Highly skewed — a small number of individuals (e.g., George W. Bush: 530 images) dominate the dataset, while most individuals have fewer than 10 images.
- Image quality: All images are standardised at 250x250 pixels in colour (RGB). Lighting, pose, and expression vary significantly across images, which is a realistic challenge for face recognition.
- Face detection rate: OpenCV Haar Cascade detector successfully detected faces in approximately 85-90% of images. The remaining 10-15% were flagged as full-image fallbacks.
- Class balance: The Top 50 subset is moderately imbalanced. This was noted as a limitation — the model may show bias towards more frequently photographed individuals.

2.4 Data Cleaning & Preprocessing

The following preprocessing pipeline was implemented:

- Face detection: Each image was processed by the OpenCV Haar Cascade detector to locate and crop the face region.
- Margin expansion: Detected face bounding boxes were expanded by 20% in each direction to include surrounding context (hair, ears, neck).
- Resize: All face crops were resized to 64x64 pixels for CNN input.
- CLAHE enhancement: Contrast Limited Adaptive Histogram Equalisation was applied to improve contrast in low-light or poorly lit images.
- Normalisation: Pixel values were normalised from [0, 255] to [0.0, 1.0] for stable CNN computation.
- Channel ordering: Images were converted from HWC (Height x Width x Channels) to CHW format for CNN processing.

.

3. Product Development & Algorithms

3.1 Algorithm Selection

The central algorithm selected for this project is a Convolutional Neural Network (CNN) for face embedding extraction, combined with cosine similarity for nearest-neighbour search. This combination is the industry-standard approach for face recognition, used in systems such as Facebook DeepFace, Google FaceNet.

A CNN was selected over alternative approaches (e.g., traditional feature extraction with SIFT/HOG + SVM, or a simple pixel-distance approach) for the following reasons:

- CNNs learn hierarchical spatial features directly from pixel data, making them significantly more robust to variations in lighting, pose, and expression than hand-crafted features.
- The embedding-based approach (extract a compact vector per face, then compare vectors) scales efficiently to large databases — searching 10,000 faces takes milliseconds using cosine similarity.
- The architecture can be extended with pretrained weights (ArcFace, FaceNet) without changing the downstream search pipeline.

3.2 CNN Architecture

The CNN was implemented entirely from scratch using NumPy — no machine learning framework (TensorFlow, PyTorch) was used. All layers including convolution, batch normalisation, max pooling, dropout, and fully connected layers were coded manually.

Layer	Output Shape	Purpose
Input	**3 × 64 × 64**	RGB face image

Conv2D(16, 3x3) + BN + ReLU + MaxPool	**16 × 32 × 32**	Low-level features
Conv2D(32, 3x3) + BN + ReLU + MaxPool	**32 × 16 × 16**	Mid-level features
Conv2D(64, 3x3) + BN + ReLU + MaxPool	**64 × 8 × 8**	High-level features
Conv2D(64, 3x3) + BN + ReLU + MaxPool	**64 × 4 × 4**	Deep abstraction
Flatten	**1024**	Vectorise
Dense(1024→256) + ReLU + Dropout(0.3)	**256**	Compress
Dense(256→128)	**128**	Embedding
L2 Normalise	**128 (unit vector)**	Cosine-ready

3.3 Similarity Search

After extracting a 128-dimensional embedding vector for a query face, cosine similarity is computed against every embedding in the database:

```
similarity(A, B) = A · B  (where ||A|| = ||B|| = 1)
```

Because both embedding vectors are L2-normalised, the dot product is equivalent to cosine similarity and ranges from 0 (completely different) to 1 (identical). Results above the user-defined threshold are returned ranked by similarity score.

3.4 Iterative Development Log

Iteration 1: Dataset Selection & Initial Setup	
What Changed	Identified LFW dataset. Downloaded Top 50 subset. Explored folder structure and image distribution. Set up Python project with Flask, OpenCV, NumPy.
Result	Successfully loaded and visualised dataset. Identified class imbalance (George W. Bush dominates with 530 images). Confirmed face detection works on sample images.
Accuracy/Impact	Face detection rate: ~85%. Project structure established.

Iteration 2: Basic UI — Upload & Display	
What Changed	Built initial Flask backend with single /search endpoint. Created basic HTML frontend with image upload functionality. System returned raw file paths as results.
Result	Working upload and display pipeline. User could upload a photo and see it displayed. No matching logic yet — results were placeholder responses.
Accuracy/Impact	UI functional. Backend receives and decodes uploaded images correctly.

Iteration 3: CNN Implementation & Face Embeddings	
What Changed	Implemented full CNN from scratch in NumPy: ConvLayer, BatchNormLayer, MaxPoolLayer, DenseLayer, DropoutLayer. Implemented L2 normalisation. Tested embedding extraction on sample faces.
Result	CNN successfully produces 128-dimensional L2-normalised embedding vectors for any face image. Verified self-similarity = 1.0. Random pair similarity ~0.0-0.3.
Accuracy/Impact	Embedding extraction working. Self-similarity = 1.000 confirmed. Architecture: 4 conv blocks + 2 FC layers.

Iteration 4: Database Indexing & Search	
What Changed	Built FaceDatabase class using NumPy arrays and JSON metadata. Indexed LFW Top 50 dataset using CNN embeddings. Implemented cosine similarity search with configurable threshold.
Result	Full search pipeline working end-to-end. Upload photo → detect face → extract embedding → search database → return ranked results with similarity percentages.
Accuracy/Impact	45 demo faces indexed initially. LFW Top 50 indexed in ~10 minutes on CPU. Search returns results in <500ms.

Iteration 5: UI Improvement — Dark Theme & Result Cards	
What Changed	Redesigned frontend with dark luxury aesthetic. Added similarity percentage badges, animated progress bars, person-grouped results view, adjustable threshold slider, CNN architecture panel.
Result	Significantly improved user experience. Results displayed as cards with face thumbnails, similarity scores, source labels. Detected faces shown as thumbnails above results.
Accuracy/Impact	UI matches commercial application standard. Person grouping correctly clusters multiple photos of same individual.

Iteration 6: Internet Search Attempt & Analysis	
What Changed	Attempted to extend the system to search the public internet for face matches (similar to PimEyes). Investigated web crawling approaches, considered scraping social media and news sites.
Result	Attempt was unsuccessful. See Section 4 for detailed analysis. System remains a local database search tool.
Accuracy/Impact	Demonstrated understanding of the technical and legal barriers. Pivoted to ArcFace pretrained model integration as alternative improvement.

Iteration 7: ArcFace Integration (V2)	
What Changed	Integrated ArcFace R50 pretrained ONNX model (trained on 5.8M images, 85k identities). Upgraded face detector to SCRFD neural detector. Added FAISS fast similarity search. Embedding dimension upgraded from 128 to 512.

Result	Dramatically improved matching accuracy. Real LFW photos now display correctly in results. Similarity scores are meaningful — same person consistently scores >0.5, different people score <0.3.
Accuracy/Impact	Recognition accuracy improved from ~random to state-of-the-art. FAISS search: <10ms for 10,000 faces. ArcFace: 512-dim embeddings.

4. Why Internet-Wide Search Was Not Achievable

During development (Iteration 6), the team investigated extending FaceTrace to perform internet-wide reverse face search — the core feature of commercial products such as PimEyes. This section documents why this attempt was unsuccessful and what would be required to replicate it.

4.1 What PimEyes Does

PimEyes operates a continuous web crawler that systematically downloads publicly accessible images from across the internet — social media profiles, news articles, forums, personal websites, and image hosting platforms. Each downloaded image is processed by a deep neural network to extract a face embedding, which is stored in a searchable index. When a user uploads a query photo, PimEyes extracts its embedding and performs a nearest-neighbour search against billions of pre-indexed embeddings.

4.2 Why Our Attempt Failed

Barrier	Explanation
No web-scale database	PimEyes has indexed billions of images crawled over years. Our system has only the LFW Top 50 dataset — a few hundred images. An internet search found no matches because we had no internet-sourced images to match against.
Legal restrictions	Systematically scraping social media platforms (Facebook, Instagram, LinkedIn) violates their Terms of Service. Major platforms block automated crawlers using CAPTCHAs, rate limiting, and IP banning.
Infrastructure scale	Crawling, downloading, and indexing even 1% of publicly available face images would require terabytes of storage, dozens of servers, and months of crawling time. This is not feasible on student hardware.
Untrained model (V1)	The initial custom CNN used randomly initialised weights — it was never trained on face data. The embeddings it produced were not semantically meaningful, so even if we had a large database, matching accuracy would have been very low.
Dynamic web content	Modern websites render content via JavaScript, making simple HTTP scraping insufficient. A full headless browser would be required to render pages before extracting images.
GDPR / Privacy law	Building a database of facial images of individuals without consent is illegal under GDPR in Ireland and the EU, regardless of whether images are publicly visible.

4.3 What Was Achieved Instead

Rather than attempting to replicate PimEyes' scale — which is the result of years of engineering and millions in infrastructure investment — the team focused on implementing the correct underlying architecture with genuine technical depth. The V2 system uses the same ArcFace model used in production face recognition systems, demonstrating that the technical pipeline is sound. The limitation is data scale, not algorithmic capability.

5. System Architecture & Deployment

5.1 End-to-End Pipeline

The complete system pipeline from user action to displayed result:

- User uploads photo via browser → JavaScript sends multipart/form-data POST to /api/search
- Flask backend decodes image bytes → converts to RGB NumPy array
- Face detection: OpenCV Haar Cascade (V1) or SCRFD neural detector (V2) locates face regions
- Face preprocessing: crop with 20% margin, resize to 64x64 or 112x112, CLAHE enhancement
- Embedding extraction: CNN forward pass produces 128-dim (V1) or 512-dim (V2) L2-normalised vector
- Database search: cosine similarity against all indexed embeddings (NumPy or FAISS)
- Results ranked by similarity, filtered by threshold, grouped by person name
- JSON response returned to frontend → rendered as cards with thumbnails and similarity bars

5.2 Technology Stack

Component	Technology	Purpose
CNN Model	NumPy (from scratch)	Face embedding extraction
Face Detector	OpenCV Haar Cascade / SCRFD	Locate face in image
Backend API	Flask (Python)	REST endpoints
Database	NumPy .npy + JSON / FAISS	Store & search embeddings
Frontend	HTML / CSS / JavaScript	User interface
Pretrained Model (V2)	ArcFace R50 via ONNX Runtime	High-accuracy embeddings
Dataset	LFW Top 50	Face image database

5.3 Deployment

The application is deployed as a local web server accessible at http://localhost:5000. The Flask development server handles concurrent requests via threading. The frontend is served as a static file by Flask, eliminating the need for a separate web server. The system was tested on Windows 11 (HP laptop, Intel CPU, no GPU) and confirmed functional.

6. Critical Evaluation

6.1 What Worked Well

- The end-to-end pipeline from image upload to ranked results works correctly and reliably.
- The CNN architecture, implemented from scratch in NumPy, correctly produces L2-normalised embedding vectors with the expected self-similarity property (similarity with self = 1.0).
- The Flask REST API is clean, well-structured, and handles edge cases (no face detected, corrupt image, empty database).
- The frontend provides a professional user experience comparable to commercial applications.
- The iterative development approach allowed systematic improvement from a basic prototype to a working product.

6.2 Limitations

- The custom CNN (V1) uses randomly initialised weights and was never trained. This means its embeddings are not semantically meaningful — matching is essentially random. This is the fundamental limitation of the initial version.
- The LFW Top 50 dataset is small. A real face search system needs millions of indexed images to be practically useful.
- Face detection using Haar Cascades fails on non-frontal faces (profile, tilted, occluded). Approximately 10-15% of images had no face detected.
- The system has no user authentication, rate limiting, or abuse prevention — it would require these before any production deployment.
- No formal accuracy metrics (TAR, FAR, ROC curve) were computed due to the random weight limitation of V1.

6.3 AI Tools Used in Development

Claude (Anthropic) was used as an AI coding assistant during development. Specifically:

- Architecture design: Claude suggested the 4-block CNN architecture and the contrastive loss approach for potential training.
- Code generation: Flask API boilerplate, frontend HTML/CSS structure, and the FAISS integration were generated with Claude assistance.
- Debugging: Claude helped diagnose the embedding dimension mismatch error when switching from V1 to V2.

Critical evaluation of AI tool usage (written independently):

Claude was effective for generating standard boilerplate code quickly, but required significant human verification and correction. It occasionally produced code that was syntactically correct but logically flawed — for example, the initial SCRFD detector output parsing was incorrect and had to be manually fixed. The AI was not used for the conceptual understanding of CNN architecture or the analysis of why internet-wide search was not achievable — these were developed through independent research and critical thinking.

7. Individual Contributions

Anjaly	Surekha
CNN architecture design & implementation	Dataset research & preprocessing
Flask REST API development	Frontend UI design & development
Face database & search pipeline	Face detection pipeline (OpenCV)
LFW dataset indexing	Weekly presentation preparation
ArcFace V2 integration	Documentation & logbook

8. References

Deng, J., Guo, J., Xue, N. and Zafeiriou, S. (2019). ArcFace: Additive Angular Margin Loss for Deep Face Recognition. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4690-4699.

LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), pp. 2278-2324.